<u>Method 2:</u>

Grammar:

File::- BeginObject Fields EndObject

BeginObject::- {
EndObject::- }

Fields::- Field1 Seperator Field2

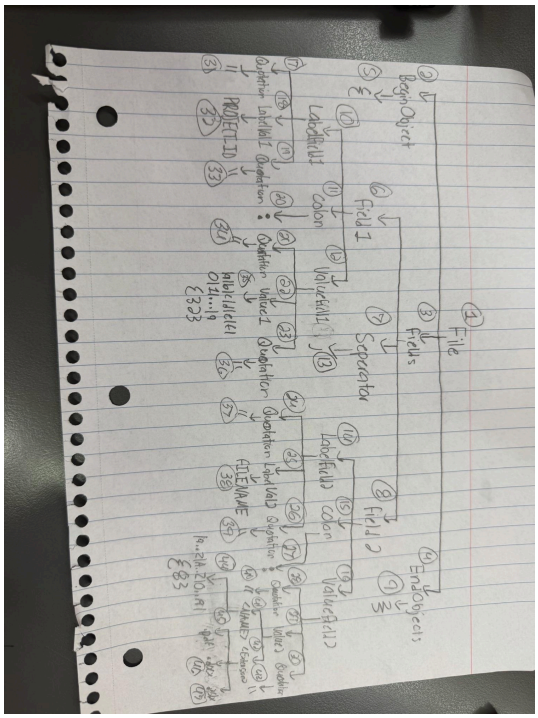
Field1::- LabelField1 Colon ValueField1
Seperator::- ,
Field2::- LabelField2 Colon ValueField2

LabelField1::- Quotation LabelVal1 Quotation
Colon::- :
ValueField1: Quotation Value1 Quotation
LabelField2: Quotation LabelVal2 Quotation
ValueField2: Quotation Value2 Quotation

Quotation::- “
LavelVal1::- PROJECT_ID
Value1::- |a|b|c|d|e|f|0|1|...|9 (32 of these)
LavelVal2::- FILENAME
Value2:::- <NAME><EXTENSION>

<NAME>::- |a…z|A…Z|0…9| (8 of these)
<EXTENSION>::- .pdf | .docx | .xslx

Derivation Tree:

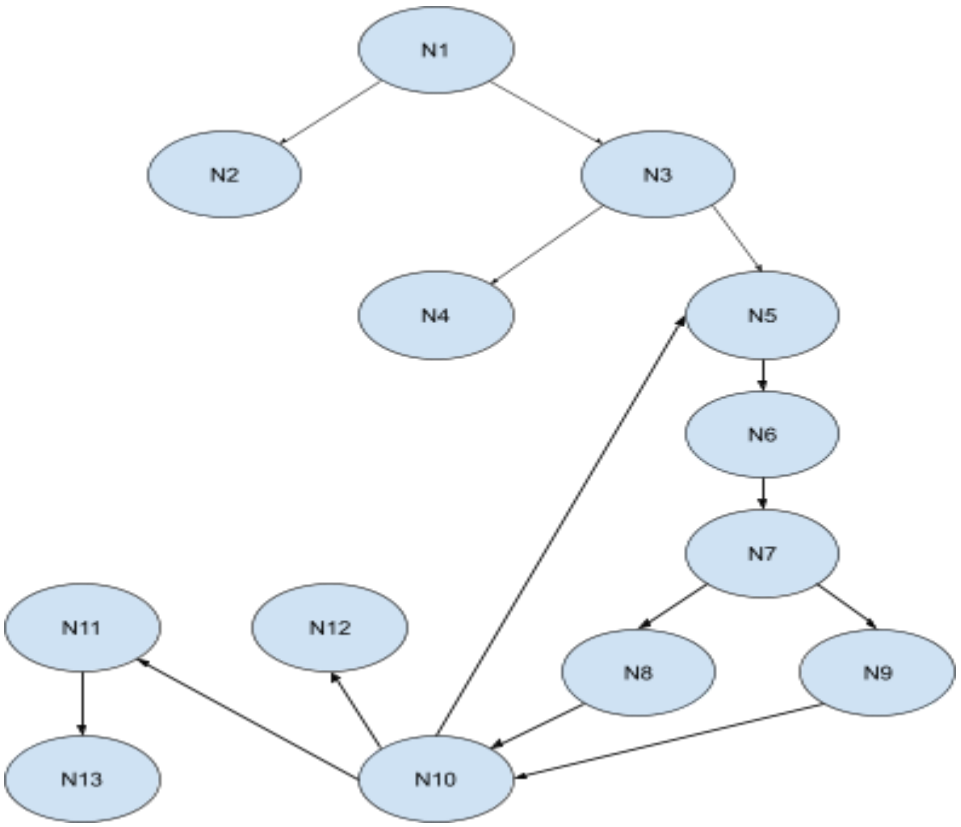


Method 3:

3.1 Flow Control Graph Nodes

To map out the execution paths, the underlying algorithm of the method was broken down into the following distinct nodes :

- **Node 1 (Start & Validation): The method `check_project_budget(project_id)` is called and the `project_id` format is validated.**
- **Node 2 (Exit): Raise `EnterpriseManagementException` due to invalid `PROJECT_ID`. -> END**
- **Node 3 (File Access): Attempt to open and read the `flows.json` file.**
- **Node 4 (Exit): Raise `EnterpriseManagementException` due to internal processing error (e.g., file not found). -> END**
- **Node 5 (Loop Condition): Check if there are unread elements in the JSON list.**
- **Node 6 (ID Match Check): Check if the "PROJECT ID" of the current list item matches the requested `project_id`.**
- **Node 7 (Flow Type Check): Mark the project as found and check if the financial movement is an "inflow" or "outflow".**
- **Node 8 (Addition): Add the "inflow" amount to the total sum.**
- **Node 9 (Subtraction): Subtract the "outflow" amount from the total sum.**
- **Node 10 (Iteration End): Move to the next item in the JSON list and return to Node 5.**
- **Node 11 (Post-Loop Check): Verify if the `project_id` was found at least once during the loop traversal.**
- **Node 12 (Exit): Raise `EnterpriseManagementException` because the `PROJECT_ID` is not in the movement file. -> END**
- **Node 13 (Success & Exit): Generate the UTC timestamp, save the calculation to a new JSON file, and return `True`. -> END**



3.2 Cyclomatic Complexity and Basic Paths

There are 6 decision points in this algorithm (Nodes 1, 3, 5, 6, 7, and 11).

M = 6 + 1 = 7

Based on this complexity, the 7 Basic Paths are defined as follows:

- **Path 1 (Invalid Input): N1 -> N2 -> END**
- **Path 2 (File Error): N1 -> N3 -> N4 -> END**
- **Path 3 (Empty List / Bypass Loop): N1 -> N3 -> N5 -> N11 -> N12 -> END**
- **Path 4 (ID Not Found): N1 -> N3 -> N5 -> N6 -> N10 -> N5 -> N11 -> N12 -> END**
- **Path 5 (Found Inflow): N1 -> N3 -> N5 -> N6 -> N7 -> N8 -> N10 -> N5 -> N11 -> N13 -> END**
- **Path 6 (Found Outflow): N1 -> N3 -> N5 -> N6 -> N7 -> N9 -> N10 -> N5 -> N11 -> N13 -> END**
- **Path 7 (Mixed Operations): N1 -> N3 -> N5 -> N6 (Match) -> N7 -> N8 -> N10 -> N5 -> N6 (No Match) -> N10 -> N5 -> N11 -> N13 -> END**

3.3 Additional Loop Testing Cases

To thoroughly test the loop constructs, the following additional test scenarios are defined:

- **0 Iterations (Bypass Loop): An empty `flows.json` file containing an empty list `[]` is provided. This verifies that the loop is safely bypassed and the method correctly raises an exception for an unfound project.**
- **1 Iteration (Single Pass): The input file contains exactly one JSON object. This tests the loop initialization and immediate termination conditions.**
- **n Iterations (Multiple Passes): The input file contains multiple JSON objects. Some objects match the requested `PROJECT_ID` with mixed inflows and outflows, while others belong to different projects. This ensures continuous loop stability and correct cumulative calculation.**